

Speculative Actions: 用于更快智能体系统的无损框架

非官方中文翻译（仅供个人学习）

Naimeng Ye*, Arnav Ahuja*, Georgios Liargkovas*, Yunan Lu*, Kostis Kaffes, Tianyi Peng
哥伦比亚大学
美国纽约州纽约市
{ny2336, aa5790, gl2902, yl4021, kk3664, tp2845}@columbia.edu

WARNING: 本文档为 arXiv:2510.04371 《Speculative Actions: A Lossless Framework for Faster Agentic Systems》的非官方中文翻译版，仅供个人学习使用；原文版权归原作者所有，请勿传播或用于商业用途。

摘要

尽管工业界和学术界对 AI 智能体的兴趣持续升温，但它们在环境中的执行通常依然很慢，从而制约了训练、评估与部署。例如，两个最先进智能体之间下一盘国际象棋，可能就需要数小时。一个关键瓶颈在于，智能体行为是按顺序展开的：每一步动作都需要一次 API 调用，而这些调用往往耗时不短。受微处理器中的推测执行以及大语言模型推理中的推测解码启发，我们提出推测动作 (*speculative actions*)：一个面向通用智能体系统的无损框架，利用更快的模型预测可能的动作，从而并行执行多个步骤。我们在三类智能体环境上评估了该框架：博弈、电商与网页搜索；同时还在操作系统环境中考察了其一个“有损”扩展。在所有场景中，推测动作都能在下一动作预测上取得可观准确率（最高可达 55%），并转化为显著的端到端延迟下降。此外，采用更强的猜测模型、top- K 动作预测、多步推测以及面向不确定性的优化，还能够进一步提升效果，为现实世界中部署低延迟智能体系统开辟出一条有前景的路径。

1 引言

由大语言模型 (LLM) 驱动的智能体，正在从一次性预测逐渐转向在复杂环境中持续运行的过程：浏览器、操作系统、游戏引擎、电商系统以及人类工作流，都是典型例子。这些环境并非附属品；它们决定了智能体能观察到什么、能做什么，通过接口和速率限制约束其推进过程，并主导端到端延迟。在实践中，智能体的行为会展开为一系列环境步骤（工具调用、MCP 服务器请求、人类参与式查询，以及进一步的 LLM 调用），而每一步都具有不可忽视的往返时间与成本。随着能力不断提升，一个新的瓶颈浮现出来：环境中的“动作响应时间” (time-to-action)。即使准确率很高，如果智能体在两步之间停顿过久，它也难以胜任交互式使用或高吞吐自动化场景。

操作系统任务 (Abhyankar et al., 2025)	深度研究 (OpenAI, 2025)	数据流水线 (Jin et al., 2025)	Kaggle 国际象棋对局 (Kaggle, 2025)
10–20 分钟	5–30 分钟	30–45 分钟	1 小时

表 1: 当前最先进 AI 智能体在不同任务/环境上的预计耗时。

如表 1 所示，AI 智能体在不同环境中完成一次运行往往需要数十分钟乃至数小时；而当强化学习或提示词优化需要进行成百上千次迭代时，这一代价还会急剧放大 (Agrawal et al., 2025)。

从根本上说，这种低效来自 API 调用的内在串行性。因此，本文提出一个简单的问题：

智能体与环境的交互，是否必须严格按顺序进行？

*共同一作

我们的回答是否定的。受微处理器中的推测执行与 LLM 推理中的推测解码启发，我们提出推测动作：一个通用框架，使智能体能够利用更快的模型预测并暂时推进最可能的下一步动作，同时让较慢的真实执行者（强大的 LLM、外部工具，或人类）在后台补上。从效果上看，智能体会对环境交互进行“预布置”（预取数据、发起安全的并行调用、准备可逆副作用），使真正关键路径从“等待”变成“验证”。当较慢的评估者确认这些猜测时，进展已经提前发生；若它们不同意，则系统仍按常规方式执行。最终得到的是一种表面上等价于顺序执行、内部却并行且机会主义的无损接口。

更具体地说，在这样的智能体中，推测动作会在环境循环里引入两个角色：

- *Actor(s)* (执行者)：权威但较慢的执行方（如能力更强的 LLM、外部 API/工具、环境自身的响应，或人类），其输出提供了正确性与副作用所依赖的真实结果。
- *Speculator(s)* (推测器)：廉价、低延迟的模型，用于预测下一步环境操作，即动作本身、其参数以及预期观测或状态变化。它可以是更小的 LLM、同一 LLM 的简化用法（减少提示词与推理步骤），或者领域启发式方法。

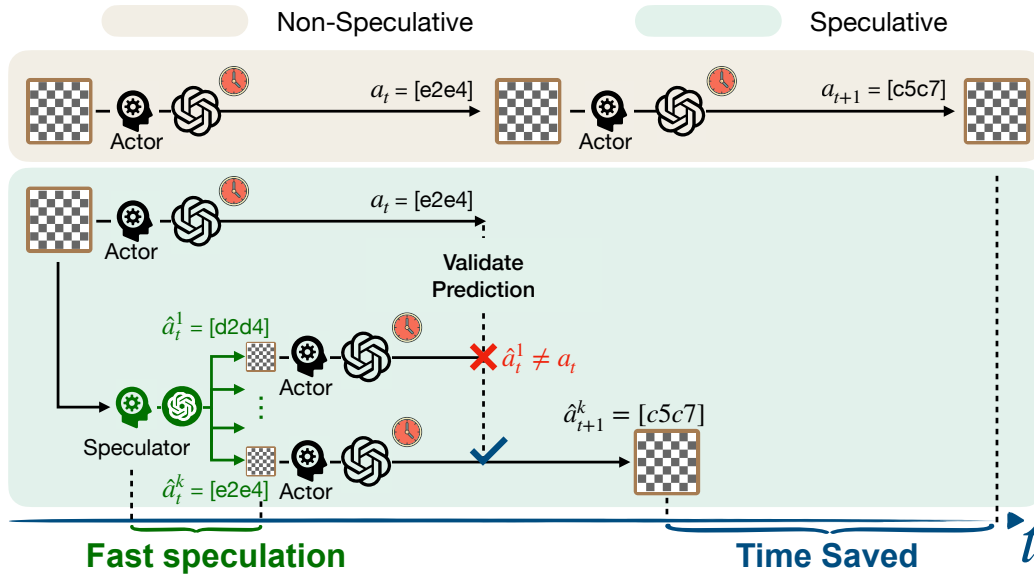


图 1: 我们在国际象棋环境中的框架示意图。当 Actor 发起一次 LLM 调用以决定下一步走法时，Speculator 会使用一个更快的模型来猜测该走法。这些猜测使后续步骤对应的 API 调用能够并行发起；一旦猜测被验证，系统便能通过并行化获得时间收益。整个过程在后台运行，因此对用户而言，这是无损的加速。

一个关键设计目标，是相对于环境原始语义保持无损性：与严格顺序执行的智能体相比，推测动作不应降低最终结果质量。我们通过以下机制实现这一点：(a) 语义守卫（Actor 在提交前确认状态转移等价），(b) 安全包络（只允许幂等、可逆或沙箱化的推测性副作用），以及 (c) 修复路径（当猜测被否定时进行回滚或补偿动作）。在许多环境中（如网页搜索流水线、结账前的购物车、沙箱中的操作系统级操作），这些模式都很自然且易于实现。

我们能否猜中智能体的下一次 API 调用？ 我们表明，在实践中，API 意图往往可以以相当合理的准确率被猜中。具体来说，我们在四类环境中展示了推测动作，每种环境都凸显了智能体延迟的不同方面：

- **回合制博弈**（如国际象棋）：Speculator 可以在等待轮到自己时预测对手的走法。参见图 1。
- **电商场景**：在与人类顾客对话时，Speculator 可以主动推断其意图（如退货），并提前安全地触发工具调用（如检查退货资格）。
- **多跳网页搜索**：在等待慢速外部调用（如 Wikipedia）返回结果时，Speculator 可以基于自身知识库猜测答案，并继续执行后续搜索查询。
- **操作系统（有损扩展）**：可逆的推测性动作可以立即响应工作负载与环境变化，在 Actor 完成确认前就提升端到端应用性能。

在这些场景中,我们都观察到显著加速:下一次 API 调用的预测准确率最高达到 55%,端到端无损加速最高达到 20%。这些结果基于一个简单的单步推测实现;若引入自适应推测等更先进技术,还能进一步提升。我们的代码已公开发布在 <https://github.com/naimengye/speculative-action>。

1.1 相关工作

微处理器中的推测 推测最初出现在计算机体系结构中,通过在结果尚未确定前提前执行指令来提高并行性 (Tomasulo, 1967); 当预测错误时再回滚, 随后它成为高性能处理器的核心机制之一 (Lam & Wilson, 1992)。针对利用微体系结构推测执行的安全漏洞, (Mambretti et al., 2019) 提出了 Speculator, 用于分析 CPU 的推测行为。

推测式程序执行 线程级推测会把原本顺序的程序片段并行执行。其前提是假定这些程序片段彼此独立、能够安全并行; 若后来发现线程之间存在数据依赖, 则必须进行回滚并解决冲突 (Estebanez et al., 2016)。最近, (Liargkovas et al., 2023) 探索了利用跟踪与隔离技术, 以推测但安全的方式乱序运行 shell 脚本。

LLM 推理中的推测解码 同样的“预测—验证”模式近来被应用到了大语言模型上。推测解码通过让一个较小的草稿模型提出 token, 再由一个更大的目标模型成批验证这些 token, 从而加速自回归推理; 正确的 token 被提交, 失败的则重新生成 (Leviathan et al., 2023; Zhang et al., 2024; Chen et al., 2023)。在推理层面, 推测也被用于加速思维链过程 (Wang et al., 2025b; a; Fu et al., 2025)。在所有这些场景中, 推测都通过并行执行可能的未来步骤及其验证, 来降低顺序流水线的延迟。

推测的其他应用 推测已广泛用于多种系统场景。例如, Speck (Nightingale et al., 2008) 用它并行化原本顺序的安全检查, AutoBash (Su et al., 2007) 则用它在隔离环境中测试配置变更。最近, Farias et al. (2024) 又提出了一种面向 GPU 的推测式策略仿真技术, 用于优化供应链系统。

受这些工作启发, 我们提出了一个面向智能体系统的推测框架, 以实现无损执行加速。我们的方法与近期对规划任务中的 LLM API 调用进行推测的研究方向一致 (Hua et al. (2024), Guan et al. (2025)), 但把这一思想推广到了**整个智能体环境**: 不仅推测 LLM 调用, 也推测内部和外部工具 API、MCP 服务器 API, 甚至人类响应。这由此形成了一个统一的智能体推测框架, 与智能体系统中新近兴起的“环境”视角和 MCP 视角高度一致, 为突破 AI 智能体训练与部署中的瓶颈提供了强有力的机制。

2 框架

智能体系统通常被建模为一个马尔可夫决策过程 (MDP) (s_t, a_t) , 其中 s_t 表示状态, a_t 表示智能体在第 t 步采取的动作。这一模型具有很高的灵活性: 动作既可以表示聊天机器人给出的回复, 也可以表示要调用的工具, 或者计算机使用型智能体点击的按钮, 等等。

从系统角度看, 我们将智能体系统中的每个动作都建模为一次 API 调用; 在返回响应前, 这种调用可能会阻塞执行。这样的抽象有两个关键优势: (1) 它精确定义了什么构成“一个动作”; (2) 它为优化系统延迟提供了一个统一框架, 后文将详细说明。值得注意的是, 这一视角与智能体系统中近期出现的 MCP 服务器发展方向是一致的 (Anthropic, 2024)。

形式化地, 在每个时间步 t , 策略 π 会把当前状态 s_t 映射为一次 API 调用:

$$(h_t, q_t) \leftarrow \pi(s_t),$$

其中 h_t 指明要调用的目标 API, q_t 是对应参数。我们记

$$\bar{a}_t \rightsquigarrow h_t(q_t) \quad a_t \leftarrow \text{await}(\bar{a}_t)$$

来表示一次异步 API 调用: 它先返回一个 *future* (待决动作), 随后在响应真正到达时再进行等待。我们用带横线的记号 (如 $\bar{a}$) 表示 *future*, 并使用缓存 $C: (h, q) \mapsto \bar{a}$, 把某次 API 调用标识映射到其待决响应。左侧的波浪箭头表示一次具有不可忽略延迟的异步调用。

随后, 系统通过状态转移函数 f 进入下一状态:

$$s_{t+1} \leftarrow f(s_t, a_t).$$

以国际象棋为例，策略 π 决定如何基于当前棋盘状态构造提示词； a_t 对应于 LLM 响应中提出的走法；而 f 则据此更新棋盘配置。注意，这里的 API 是 LLM 调用本身，而它的响应才是走法 a_t 。

这一表述可以覆盖广泛的具体实现：

- **LLM 调用**：智能体内部每一次 LLM 调用，都可以被视为一个动作。
- **工具 / MCP 服务器调用**：对内部或外部工具的每一次真实调用，也可视为动作，例如终端访问、网页搜索、深度研究 API、天气 API，或 browser-use MCP 等。
- **将人类视为 API 的调用**：进一步地，人类响应本身也可以被抽象成 API 调用，而其延迟往往比自动化工具还要更长。

基于这一抽象，执行智能体系统时的根本瓶颈就很清楚了：每一次 API 调用都必须完成，下一次调用才能发起。为打破这种串行依赖，我们提出在等待真实响应 a_t 的同时，利用更快的模型去推测一组 API 响应 $\{\hat{a}_t\}$ 。这样一来，第 $t+1$ 步对应的推测性 API 调用就能并行发起。在时刻 t ，若 API 调用 (h_t, q_t) 已经能在缓存中找到（即缓存命中），系统就可以跳过真实调用，只等待与该调用对应的待决动作返回（如果它尚未返回）。形式化算法如算法 1 所示。

算法 1 具有 k 路并行下一步调用的推测动作

输入：初始状态 s_0 、时域长度 T 、转移函数 f 、策略 π 、预测器 $\hat{g}$ 、缓存 $\mathcal{C}$ 。我们用 $\bar{a}$ 表示待决动作。

```

1: for  $t = 0$  to  $T - 1$  do
2:   策略:  $(h_t, q_t) \leftarrow \pi(s_t)$ 
3:   if  $(h_t, q_t) \in \mathcal{C}$  then ▷ 缓存命中
4:      $\bar{a}_t \leftarrow \mathcal{C}[(h_t, q_t)]$ 
5:      $a_t \leftarrow \text{await}(\bar{a}_t)$  ▷ 若该待决动作尚未返回，则等待
6:      $s_{t+1} \leftarrow f(s_t, a_t)$ 
7:     continue
8:   end if
9:   Actor: 发起真实请求（返回 future）:  $\bar{a}_t \rightsquigarrow h_t(q_t)$ 
10:  Speculator:  $\{\hat{a}_t^{(i)}\}_{i=1}^k \leftarrow \text{await}(\hat{g}(s_t, (h_t, q_t)))$  ▷ Actor 与 Speculator 并行执行
11:  for  $i = 1$  to  $k$  do ▷ 针对每个猜测执行一步推测展开
12:     $\hat{s}_{t+1}^{(i)} \leftarrow f(s_t, \hat{a}_t^{(i)})$ 
13:     $(\hat{h}_{t+1}^{(i)}, \hat{q}_{t+1}^{(i)}) \leftarrow \pi(\hat{s}_{t+1}^{(i)})$ 
14:    预启动:  $\bar{\hat{a}}_{t+1}^{(i)} \rightsquigarrow \hat{h}_{t+1}^{(i)}(\hat{q}_{t+1}^{(i)})$  ▷ 返回待决动作，因此非阻塞
15:     $\mathcal{C}[(\hat{h}_{t+1}^{(i)}, \hat{q}_{t+1}^{(i)})] \leftarrow \bar{\hat{a}}_{t+1}^{(i)}$  ▷ 缓存推测得到的待决动作
16:  end for
17:  等待 Actor 返回已解析的  $a_t$ :  $a_t \leftarrow \text{await}(\bar{a}_t)$ 
18:   $s_{t+1} \leftarrow f(s_t, a_t)$ 
19: end for

```

由此带来的加速依赖于两个关键假设：

假设 1 (推测准确性). 推测模型 $\hat{g}$ 能够足够准确地猜测当前步的响应 a_t ，使得由此隐含的下一调用 $(h_{t+1}, q_{t+1}) = \pi(f(s_t, \hat{a}_t))$ 以非零概率 $p > 0$ 与真实的下一调用一致。

正如后文所示，这一假设在实践中往往成立，因为 API 响应通常具有一定可预测性。

假设 2 (并发、可逆的预启动). 多个 API 调用可以并发发起；而那些与实际轨迹不一致的预启动调用，不会产生对外可见的副作用（或其副作用可以回滚）。

在实践中，对于许多外部 API（如网页搜索、OpenAI LLM 查询、邮箱查找等），只要流量适中，这一假设通常是成立的。对于自托管 LLM，由于持续批处理（continuous batching）的存在，并发调用带来的额外成本也很小。

于是我们可以得到如下结果（证明延后至附录）：

命题 1. 在假设 1-2 成立的前提下，设在每一步上，推测分支以概率 p 推出正确的下一调用 (h_{t+1}, q_{t+1}) ，并且对 $t \in [1, T-1]$ 独立成立。设 $\hat{g}$ 的延迟服从 $\text{Exp}(\alpha)$ ，真实 API 调用的延

迟服从 $\text{Exp}(\beta)$, 且 $\beta < \alpha$ 。所有延迟与猜测相互独立。再假设状态转移 f 和 API 参数构造 π 的开销都可忽略不计。则算法 1 的期望运行时间 (记为 $\mathbb{E}[T_s]$) 与顺序执行期望运行时间 (记为 $\mathbb{E}[T_{\text{seq}}]$) 之比为

$$\frac{\mathbb{E}[T_s]}{\mathbb{E}[T_{\text{seq}}]} = 1 - \frac{1}{T} \frac{\alpha}{\alpha + \beta} \left[\frac{(T-1)p}{1+p} + \frac{p^2}{(1+p)^2} - \frac{p^2}{(1+p)^2} (-p)^{T-1} \right] \xrightarrow{T \rightarrow \infty} 1 - \frac{p}{1+p} \cdot \frac{\alpha}{\alpha + \beta}$$

命题 1 的证明见节 A。命题 1 表明, 在理想情况下若 $p = 1$ 且 $\alpha = \infty$, 端到端延迟下降的上界为 50%; 但这一上界还可以通过下文的多步扩展继续提升。

扩展。 算法 1 只是推测动作思想的一个简单示范。举例而言, 我们可以自然地把它扩展为多步推测: Speculator 不仅预测下一步, 还能向前预测多达 s 步。这样就得到一个具有更深 rollout 的树搜索结构。它还可以进一步与自适应推测结合: Speculator 不再对 a_t 均匀地产生 k 个猜测, 而是同时为每个猜测估计置信度 (例如通过 LLM 提示或不确定性量化方法)。随后, 最有希望的分支就可以像束搜索那样被继续展开。上述思路共同表明, 推测动作具有丰富的延展空间, 我们将其留待未来进一步深入研究。尽管算法 1 非常简单, 但后续四个用例的结果已经相当鼓舞人心。

成本—延迟权衡。 发起更多推测性 API 调用 (例如增大 k) 会提升准确率, 但如果定价依据是调用次数或 token 数, 那么成本也会随之上升。尽管成本不是本文重点, 我们仍在附录 B 中给出了实验分析。对于自托管 LLM, 这一成本增长在很大程度上会被批处理机制抵消。

副作用与安全性。 推测会执行一个假设的下一动作 $\hat{a}_{t+1}$, 而这个动作可能是错的, 因此安全性要求系统能够先模拟, 再提交或回滚。在国际象棋这类领域中, 回滚很容易; 在其他场景里, 覆盖写入也较容易处理 (例如操作系统调优)。但很多系统都涉及不可逆或对外可见的效果 (如删除记录、下单付款), 这时朴素推测就会带来危害。因此, 推测必须被限制在误判可逆的情形中, 例如通过分叉、快照恢复, 或前滚式修复 (如退款/补发) 来处理。

3 环境

下面我们将在三类环境中实例化推测动作: 国际象棋、电商对话, 以及多跳网页搜索。选择这三类环境, 是为了分别突出不同的延迟瓶颈 (推理、工具/API 往返, 以及信息检索)。在每种情况下, 我们都将一个快速的 Speculator 与一个缓慢的 Actor 配对, 并实现算法 1。我们报告预测准确率以及端到端节省的时间。

3.1 国际象棋环境

我们在竞争性多智能体博弈场景中展示该框架的有效性, 并以国际象棋这一典型回合制任务作为例子。在标准对弈中, 分析过程是严格串行的: 每位玩家都只能在对手完成其回合之后才开始思考。这样的串行化会引入大量空闲等待时间。尤其当双方都依赖计算量较大的推理模型时, 一盘棋的实际耗时可能被拉长到数小时。我们的框架通过推测式并行分析放宽了这一限制, 使玩家可以提前预判并准备对手最可能的走法。我们将表明, 这能显著缩短整盘对局的总时长。

3.1.1 实现

我们基于 TextArena (Guertler et al., 2025) 实现该框架; 它为 LLM 驱动的智能体提供了标准化的博弈接口。

推测流水线。 在回合 t , 游戏状态 s_t 对应当前棋盘局面。当前行棋玩家发起一次 API 调用 h_t , 其参数 q_t 由 s_t 与一条引导推理的提示词共同构成。此时玩家 P 负责落子, 玩家 Q 等待。我们的框架按如下方式运行:

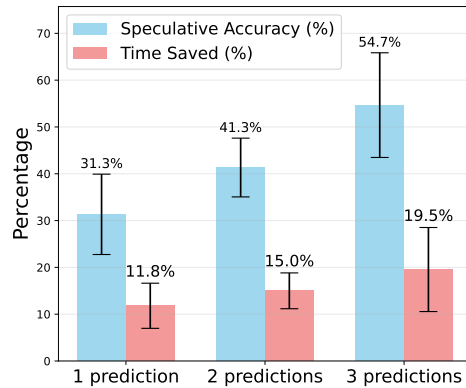


图 2: 在 30 个步数、共 5 次运行上的节省时间比例与正确预测比例。

- **当前行棋玩家 P** : 玩家接收 s_t , 并向智能体发起 API 调用 h_t , 其中参数 $q_t = (s_t, \text{prompt})$ 。该调用返回下一步走法 $a_t = h_t(q_t)$ 。由于需要深度、充分的推理, 这一步通常具有较高延迟。

- **未轮到行动的玩家 Q** :

1. **预测阶段**。Speculator 同样接收棋盘状态 s_t , 并发起一次 API 调用 $\hat{h}_t$, 但使用的是针对速度而非深度优化的提示词。它返回按置信度排序的 top- k 走法预测 $\hat{a}_t^{(1)}, \hat{a}_t^{(2)}, \dots, \hat{a}_t^{(k)}$ 。
2. **并行计算**。对每个预测走法 $\hat{a}_t^{(i)}$, 未行动玩家 Q 会立刻启动一个并行进程, 分析下一步应对走法 $\hat{a}_{t+1}^{(i)} = h_{t+1}(\hat{s}_{t+1}^{(i)}, \text{prompt})$, 其中 $i \in \{1, \dots, k\}$, 而 $\hat{s}_{t+1}^{(i)} = f(s_t, \hat{a}_t^{(i)})$ 表示将预测动作 $\hat{a}_t^{(i)}$ 作用于 s_t 后得到的下一状态。
3. **验证**。当当前行棋玩家 P 完成推理并返回其走法 a_t 后, 我们立即检查该走法是否与某个预测走法 $\hat{a}_t^{(1)}, \hat{a}_t^{(2)}, \dots, \hat{a}_t^{(k)}$ 相匹配。
4. **提交或重启**。若存在匹配, 我们就提交相应的推测分支, 直接推进到 $s_{t+1} = f(s_t, a_t)$, 并终止其他线程。若没有匹配, 则丢弃所有推测分支, 并继续正常计算玩家 Q 的正式走法 $a_{t+1} = h_{t+1}(f(s_t, a_t), \text{prompt})$ 。

这一流水线是无损的: 最终轨迹与不使用推测的对弈完全一致, 但由于推理被并行化, 整体耗时更短。

智能体配置。我们发现, 对 Speculator 与 Actor 使用同一个模型、但搭配不同提示词, 可以在保持推测快速的同时最大化预测准确率。因此, 在实验中, Actor 采用高推理强度的 GPT-5, 每一步走法都通过一次 API 调用产生; Speculator 同样使用 GPT-5, 但配置为低推理强度, 并配合一条专为快速走法预测而非穷尽分析设计的系统提示词。

3.1.2 结果

我们从节省时间与预测准确率两个角度评估该框架。

我们跟踪两个指标: (i) 预测准确率, 即在多少比例的回合中, 至少有一个推测结果与真实走法一致; (ii) 节省时间, 定义为 $(T_{\text{seq}} - T_s)/T_{\text{seq}}$, 其中 T_s 和 T_{seq} 分别表示推测执行与顺序执行所需时间。

预测越多, 节省时间和准确率越高。图 2 展示了 30 步对弈下的结果。与顺序对弈相比, 我们的框架始终能够缩短执行时间, 而且随着推测候选数增加, 收益也随之增加。在 5 次运行中, 使用 3 个预测时, 平均节省时间为 19.5%, 平均预测准确率为 54.7%。

博弈中智能体调用的随机性。图 2 中的方差反映了真实 API 调用下的实际对局动态。即便预测正确, 节省的时间也不固定: 如果落子后的局面存在显而易见的应对, 那么无论是否推测, 后续计算本来就会很快, 因此加速效果有限。只有当预测通向需要深度分析的局面时, 才会带来显著收益。因此, 推测的效果同时取决于预测准确率以及所产生局面的复杂度。

3.2 电商环境

除了竞争性博弈之外, 电商中的顾客—智能体交互也是一个非常贴近现实的场景, 在这里推测动作能够显著改善性能。典型工作流中, 顾客通过聊天界面提交请求, 然后等待智能体响应。当智能体需要顺序调用多个 API 时——例如处理退货可能需要先获取订单信息、再验证每件商品的退货资格、最后发起退货——由此产生的延迟会明显损害用户体验。相反, 如果某些 API 调用能够被正确推测并提前执行, 那么在用户请求真正到达时, 智能体就可以立即返回结果, 从而大幅降低延迟, 使交互体验更加顺滑。我们在 τ -bench 的零售环境上评估这一设置 Yao et al. (2024)。

3.2.1 实验设置

推测流水线 在该场景下, 当前状态 s_t 定义为截至回合 t 的对话历史, 而 h_t 表示为回答用户查询所需调用的 API (例如 `get_user_details`、`get_order_details`)。我们的 Speculator 将预测:

1. 用户的查询 $\hat{a}_t$;

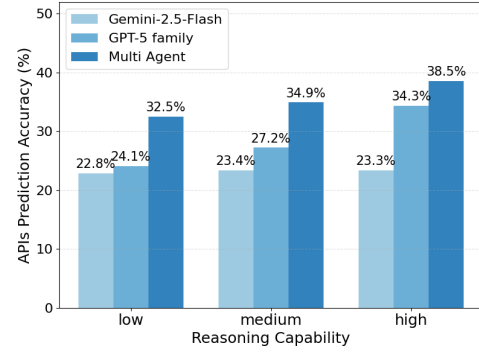
2. 在当前状态 s_t 与第 1 步得到的用户查询预测共同条件下，对应的目标 API 调用及其参数 $(\hat{h}_{t+1}^{(i)}, \hat{q}_{t+1}^{(i)})$ ，其中 $i \in \{1, \dots, k\}$ 。由于每个回合需要的 API 调用数并不固定，Speculator 还必须同时预测 k 。

智能体配置 我们测试了多种 Speculator 模型：OpenAI GPT 系列（gpt-5-nano、gpt-5-mini、gpt-5）以及 Google Gemini 系列（gemini-2.5-flash），并设置不同的推理预算（1024、2048、4096 tokens）。已有工作 Jiang et al. (2023); Chen et al. (2025) 表明，异构 LLM 集成通常优于单一模型，而且多个模型可以在相同时间预算下并行执行。受此启发，我们评估了两种 Speculator 配置：(i) 单模型设置，即推测仅依赖一个模型；(ii) 多模型设置，即让能力与推理预算相近的模型并行运行（例如 gpt-5-nano 搭配低预算 Gemini，gpt-5-mini 搭配中预算 Gemini，以及 gpt-5 搭配高预算 Gemini）。随后，它们的预测会被汇总为一个共享的推测动作候选集合。在运行时，当用户模拟器给出真实输入后，Actor 会将推测得到的 API 调用与真实所需 API 进行比较。正确的预测会被立即提交，从而消除延迟；错误的预测则被安全丢弃，不影响正确性。

评估 我们使用 **API 预测准确率** 来衡量性能，即推测出的 API 调用中，有多少比例与解决用户问题所需的真实 API 调用相匹配。该指标直接反映了用户在多少比例的回合中能够立刻得到响应，而无需等待 API 真正执行。换言之，更高的预测准确率会直接转化为更大的时间收益。

3.2.2 结果

图 3 表明，Speculator 能够正确预测 22% 到 38% 的 API 调用。随着模型能力增强，准确率会提升；同时，多模型配置也稳定优于单模型推测。更重要的是，低预算模型完成一次推测只需 2-3 秒（依据 LLM API providers leaderboard¹），远低于平均约 30 秒的用户打字时间（按每分钟 40 个词估算）。这意味着，在约三分之一的回合中，智能体都可以比顺序执行更快地响应，而无需等待 API 执行完成。上述结果说明，在工具密集型环境中，推测可以把原本明显滞后的用户体验，转变为接近实时的交互。



3.3 HotpotQA 环境

我们还在 HotpotQA 上进一步评估了该框架。在这一任务中，主要性能瓶颈来自信息检索延迟。

具体来说，智能体需要通过顺序调用 Wikipedia API 来回答多跳问题 Yang et al. (2018)。这种工具调用模式与现实世界中每轮往返都具有较高网络延迟的智能体 workflows 非常相似。我们采用节 2 中的框架：当真实 API 调用仍在执行时，Speculator 预测可能的 Wikipedia 内容。借助这种并行性，智能体不必阻塞等待 API 返回，而可以基于暂定信息继续推理。

图 3: 不同 Speculator 模型在不同推理能力设置下的 API 预测准确率。

3.3.1 实验设置

我们在 ReAct Yao et al. (2023) 的基础上构建框架；ReAct 会交替进行思维链推理与工具使用。

推测流水线。在这个场景中，状态 s_t 由完整的推理轨迹历史以及已经检索到的信息（即 API 响应）组成。在每个时间步，Actor 接收当前状态 s_t ，选择一个 API 调用 $h_t \in \{\text{Search}(), \text{Lookup}(), \text{Finish}()\}$ 及其对应参数 q_t ，例如 $\text{Search}(\text{entity})$ 。调用 $h_t(q_t)$ 会返回一个响应 a_t ，通常提供与查询实体相关的信息。我们的推测框架按如下方式工作：

1. Speculator 预测 API 调用响应 $\hat{a}_t^{(i)}$ ，从而得到预测的下一状态 $\hat{s}_{t+1}^{(i)} = f(s_t, \hat{a}_t^{(i)})$ ，其中 $i \in \{1, \dots, k\}$ 。
2. 基于这些状态，Actor 会生成推理轨迹，并进一步决定下一次 API 动作 $(\hat{h}_{t+1}^{(i)}, \hat{q}_{t+1}^{(i)})$ ，其中 $i \in \{1, \dots, k\}$ 。

¹<https://artificialanalysis.ai/leaderboards/providers>

评估。我们通过预测得到的 API 决策 ($\hat{h}_{t+1}, \hat{q}_{t+1}$) 的准确率来评估推测流水线的有效性。具体而言，我们将预测调用与在真实响应 a_t 下得到的真实调用 (h_{t+1}, q_{t+1}) 进行比较。我们采用严格匹配标准：只有当 $\hat{h}_{t+1} = h_{t+1}$ 且 $\hat{q}_{t+1} = q_{t+1}$ 时，才计为正确。这个严格标准能够准确反映推测是否真的带来了有效推进；即便只是参数表达上的细微差别（如同义词或词序不同），也会被判定为不匹配。

智能体配置。我们在三种 Speculator 模型上评估推测准确率：GPT-5-nano、GPT-4.1-nano 和 Gemini-2.5-flash。对每种模型，我们都测量 top- k 预测准确率，其中 $k \in \{1, 3\}$ 。

3.3.2 结果

图 4 表明，尽管我们采用了严格匹配标准，Speculator 在 top-3 预测下仍能以最高 46% 的概率预测中真实 API 调用。与 top-1 相比，top-3 的准确率有明显提升，这说明只要适度增加推测宽度，就能带来显著的准确率收益。我们的推测机制之所以有效，是因为它可以在原本等待 API 返回的空闲时间里，预先计算后续推理路径。

模型模式。我们观察到，不同 Speculator 在 API 决策上存在较大差异，这主要来自措辞层面的不同——有些模型的调用表述更简洁，有些则更倾向于过度具体化。有趣的是，更强的模型有时反而准确率更低，因为它们更容易给出多样化且强上下文依赖的查询（例如“List of Nobel laureates in physics 1970s”与“1970s Nobel Prize Physics winners list”），而在严格匹配标准下，这些差异都会受到惩罚。相比之下，较弱的模型往往输出更简单、也更可预测。

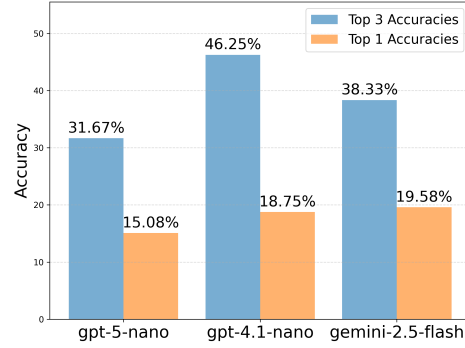


图 4: 以 gemini-2.5-flash 作为 Actor 时的准确率。与只预测单个动作相比，推测多个动作 ($k = 3$) 会得到更高准确率。

4 超

越无损推测：操作系统超参数调优环境

到目前为止，我们的实验都聚焦于无损推测：推测动作必须逐步验证之后才能提交。现在我们转向一个放宽这一逐步约束的有损场景。在操作系统这类对延迟极其敏感的环境中，等待一个强大但缓慢的 Actor（需要 10–15 秒进行思考）可能会让系统长时间停留在退化状态。我们的框架使用一个快速的 Speculator 立即施加调整，在 Actor 还在思考时就改善实时性能。其安全性由“最后写入者获胜”（last-write-wins）机制保证——Actor 的最终决策会直接覆盖任何推测动作，从而无需复杂回滚。这种方法加快了收敛，并提升了响应速度；我们在 sysbench cpu 基准上评估这一点，该基准是一个 CPU 密集型工作负载 (Kopytov, 2020)。

4.1 实验设置

我们调节 Linux 完全公平调度器 (CFS) 的一个参数 `min_granularity`，它控制任务的最小时片长度。这个参数会强烈影响调度性能：较小的时片可以降低延迟，但可能损害吞吐；这是一个经典权衡。已有工作 (Liargkovas et al., 2025) 证明，基于 LLM 的智能体可以优化这一参数；在此基础上，我们进一步引入了推测式控制回路。

我们的系统由一个快速的 Speculator 和一个缓慢的 Actor 构成。Speculator 每秒根据最新性能指标提出一次有界的参数更新。相比之下，Actor 每 10–15 秒响应一次；它会分析近期由 Speculator 产生的压缩版（测量值，动作）序列。当 Actor 的响应到达时，其选定设置会被立刻应用，并用来更新 Speculator 的上下文，从而保证后续快速步骤建立在经过验证的叙述之上，而不是不断漂移。

评估 我们评估三种系统：

1. **仅 Actor**：较慢，但更具审慎分析能力（10–15 秒一个决策周期）；
2. **仅 Speculator**：较快（1 秒一个决策周期），但分析不够深入；

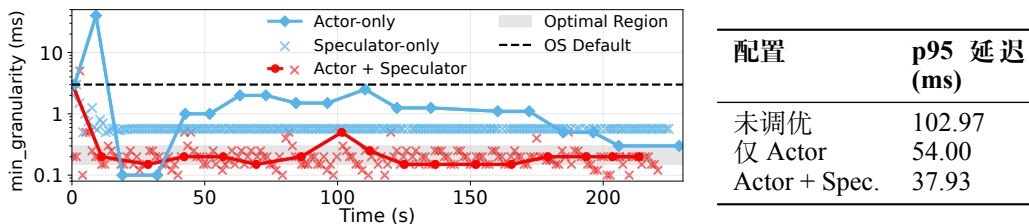


图 5: (左) **Speculator-Actor**、仅 **Speculator** 与仅 **Actor** 的收敛过程对比。Speculator 缩短了系统在糟糕设置上的探索时间。仅 **Speculator** 虽然稳定得很快，但最终值更差。(右) 一个 20 秒调优实验中的平均 p95 延迟，说明快速反应能立刻带来性能收益（见 §B.3.4）。数值越低越好。

3. Speculator-Actor：在 Actor 决策之间插入推测更新的联合系统。

我们关心三个方面：(i) 响应时间，(ii) 收敛到最优设置的速度，以及 (iii) 是否能够避免“仅 Speculator”基线容易陷入的局部最优。

4.2 结果

Speculator 缓解了慢响应带来的性能恶化 正如图 5 (右) 所示，Speculator 显著改善了系统的反应速度。在恢复阶段，完整的 Speculator-Actor 系统将平均 p95 延迟维持在 37.93 ms。这相较于仅 Actor 的基线有明显提升：后者不得不在较差配置下停留更久，平均延迟达到 54.00 ms。Speculator 的快速纠偏避免了系统长期停留在高延迟状态（初始值为 102.97 ms），因此即便 Actor 仍在较慢地思考，也能立刻带来收益（完整实验细节见 §B.3.4）。

Speculator 加速收敛到最优值 Speculator 的高频更新也显著加快了收敛。如图 5 (左) 所示，Speculator-Actor 系统大约在 10–15 秒内就能到达一个最优设置（例如 `min_granularity` = 0.2 ms）。相比之下，仅 Actor 智能体大约需要 200 秒——慢了约 20 倍——并且在很长一段时间内都被困在高度次优的状态中（例如延迟 > 120 ms）。Speculator 的快速探索为 Actor 提供了更丰富的性能地形信息，使其能够更快地把系统从这些病态区域中拉出来。

仅 Speculator 很快，但并不最优 图 5 还显示，尽管仅 Speculator 系统响应很快，但它会过早收敛到一个次优配置：`min_granularity` 为 0.55 ms (36.24 ms 延迟)。这一结果明显劣于完整 Speculator-Actor 系统达到的 0.2 ms (30.26 ms 延迟)。没有 Actor 的指导，Speculator 缺乏足够的推理深度，无法跳出这一局部最优。

联合的 Speculator-Actor 系统将快速适应与策略性指导结合起来，同时实现了高响应性与最优稳态性能。

5 结论

本文提出了**推测动作 (Speculative Actions)**：一个通过打破交互循环严格串行性来加速通用智能体环境的无损框架。与 LLM 推理中的推测解码不同，我们的方法将推测推广到完整的智能体-环境交互谱系之中——把每一步，无论是 LLM 调用、工具调用、MCP 请求还是人类响应，都视为可以被预测与并行化的 API 调用。通过将快速的 *Speculator* 与一个缓慢但权威的 *Actor* 配对，该框架使智能体能够并行地预判并准备最可能发生的下一步动作，把原本纯粹的等待时间转化为有效计算。我们对期望加速效果给出了形式化分析，表明在简单假设下理论上的延迟下降最高可达 50%；同时，我们还在四类代表性环境中实例化了这一框架。实验上，我们观察到显著的总耗时与延迟下降，验证了推测动作能够在不牺牲正确性的前提下稳定提升效率。特别地，尽管大多数环境采用的是严格的无损验证过程，我们的系统级实验表明，一旦适度放宽这一限制，推测也可以扩展到需要快速探索性更新的自适应控制场景中。

更广泛地看，推测动作揭示了一条面向现代智能体平台的系统级设计原则：环境交互中的机会主义并行性。未来工作可以从多步与不确定性感知推测、层次化的 Actor-Speculator 架构，或与强化学习及环境模拟器更紧密的结合等方向继续推进。总之，这些方向共同指向一类通用、实时的智能体系统：在这样的系统中，推测将成为高效决策的一等公民机制。

参考文献

- Reyna Abhyankar, Qi Qi, and Yiyang Zhang. Osworld-human: Benchmarking the efficiency of computer-use agents. *arXiv preprint arXiv:2506.16042*, 2025.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, November 2024. Accessed: 2025-09-24.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Zhijun Chen, Jingzheng Li, Pengpeng Chen, Zhuoran Li, Kai Sun, Yuankai Luo, Qianren Mao, Ming Li, Likang Xiao, Dingqi Yang, Yikun Ban, Hailong Sun, and Philip S. Yu. Harnessing multiple large language models: A survey on llm ensemble, 2025. URL <https://arxiv.org/abs/2502.18036>.
- Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pp. 1–14, July 2019. URL <https://www.flux.utah.edu/paper/duplyakin-atc19>.
- Alvaro Estebanez, Diego R. Llanos, and Arturo Gonzalez-Escribano. A survey on thread-level speculation techniques. *ACM Comput. Surv.*, 49(2), June 2016. ISSN 0360-0300. doi: 10.1145/2938369. URL <https://doi.org/10.1145/2938369>.
- Vivek Farias, Joren Gijsbrechts, Aryan Khojandi, Tianyi Peng, and Andrew Zheng. Speeding up policy simulation in supply chain rl. *arXiv preprint arXiv:2406.01939*, 2024.
- Yichao Fu, Rui Ge, Zelei Shao, Zhijie Deng, and Hao Zhang. Scaling speculative decoding with lookahead reasoning. *arXiv preprint arXiv:2506.19830*, 2025.
- Yilin Guan, Wenyue Hua, Qingfeng Lan, Sun Fei, Dujian Ding, Devang Acharya, Chi Wang, and William Yang Wang. Dynamic speculative agent planning, 2025. URL <https://arxiv.org/abs/2509.01920>.
- Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. Textarena, 2025. URL <https://arxiv.org/abs/2504.11442>.
- Wenyue Hua, Mengting Wan, Shashank Vadrevu, Ryan Nadel, Yongfeng Zhang, and Chi Wang. Interactive speculative planning: Enhance agent efficiency through co-design of system and user interface, 2024. URL <https://arxiv.org/abs/2410.00079>.
- Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-blender: Ensembling large language models with pairwise ranking and generative fusion. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14165–14178, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.792. URL <https://aclanthology.org/2023.acl-long.792/>.
- Tengjun Jin, Yuxuan Zhu, and Daniel Kang. Elt-bench: An end-to-end benchmark for evaluating ai agents on elt pipelines. *arXiv preprint arXiv:2504.04808*, 2025.
- Kaggle. Game arena. <https://www.kaggle.com/game-arena>, 2025. Accessed: 2025-09-21.
- Alexey Kopytov. Sysbench: Scriptable benchmark tool. <https://github.com/akopytov/sysbench>, 2020. Accessed: 2025-09-22.

-
- Monica S Lam and Robert P Wilson. Limits of control flow on parallelism. *ACM SIGARCH Computer Architecture News*, 20(2):46–57, 1992.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.
- Georgios Liargkovas, Konstantinos Kallas, Michael Greenberg, and Nikos Vasilakis. Executing shell scripts in the wrong order, correctly. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems*, pp. 103–109, 2023.
- Georgios Liargkovas, Vahab Jabrayilov, Hubertus Franke, and Kostis Kaffes. An expert in residence: Llm agents for always-on operating system tuning. In *Proceedings of the NeurIPS 2025 Workshop on Machine Learning for Systems (MLForSys)*, San Diego, CA, USA, December 2025. NeurIPS. Accepted paper.
- Andrea Mambretti, Matthias Neugschwandtner, Alessandro Sorniotti, Engin Kirda, William Robertson, and Anil Kurmus. Speculator: a tool to analyze speculative execution attacks and mitigations. In *Proceedings of the 35th Annual Computer Security Applications Conference*, pp. 747–761, 2019.
- Edmund B Nightingale, Daniel Peek, Peter M Chen, and Jason Flinn. Parallelizing security checks on commodity hardware. *ACM SIGARCH Computer Architecture News*, 36(1):308–318, 2008.
- OpenAI. Introducing deep research. <https://openai.com/index/introducing-deep-research/>, 2025. Accessed: 2025-09-24.
- Ya-Yunn Su, Mona Attariyan, and Jason Flinn. Autobash: improving configuration management with operating system causality analysis. *ACM SIGOPS Operating Systems Review*, 41(6):237–250, 2007.
- Robert M Tomasulo. An efficient algorithm for exploiting multiple arithmetic units. *IBM Journal of research and Development*, 11(1):25–33, 1967.
- Jikai Wang, Juntao Li, Jianye Hou, Bowen Yan, Lijun Wu, and Min Zhang. Efficient reasoning for llms through speculative chain-of-thought, 2025a. URL <https://arxiv.org/abs/2504.19095>.
- Zhihai Wang, Jie Wang, Jilai Pan, Xilin Xia, Huiling Zhen, Mingxuan Yuan, Jianye Hao, and Feng Wu. Accelerating large language model reasoning via speculative search, 2025b. URL <https://arxiv.org/abs/2505.02865>.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018. URL <https://arxiv.org/abs/1809.09600>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, June 2024. URL <http://arxiv.org/abs/2406.12045>. arXiv:2406.12045.
- Chen Zhang, Zhuorui Liu, and Dawei Song. Beyond the speculative game: A survey of speculative execution in large language models. *arXiv preprint arXiv:2404.14897*, 2024.

A 命题 1 的证明

证明. **基线**。在顺序执行中, T 个步骤中的每一步都需要调用一次真实模型 h , 其平均延迟为 $1/\beta$ 。因此

$$E[T_{\text{seq}}] = \frac{T}{\beta}.$$

每次命中时节省的期望时间。考虑两个连续步骤 $(t, t+1)$ 。在基线中, 总完成时间为 $R = B + C$, 其中 $B, C \sim \text{Exp}(\beta)$ 分别表示第 t 步与第 $t+1$ 步的延迟。使用推测时, 我们会在第 t 步期间发起 $A \sim \text{Exp}(\alpha)$ 。如果猜测正确, 那么一旦 A 或 B 中任意一个先完成, 就可以发起第 $(t+1)$ 步的调用 C , 于是这个两步块的完成时间为

$$S = C + \min\{A, B\}.$$

因此, 当猜测正确时, 我们节省的期望时间为

$$R - S = (B - A)_+,$$

其中 $(x)_+ = \max\{x, 0\}$ 。

由 A, B 相互独立可得

$$\mathbb{E}[(B - A)_+] = \int_0^\infty \int_0^b (b - a) \alpha e^{-\alpha a} \beta e^{-\beta b} da db = \frac{\alpha}{\beta(\alpha + \beta)}.$$

命中次数的期望。记到第 n 轮为止命中的期望次数为 S_n , 即

$$\mathbb{E}[\text{到第 } n \text{ 轮为止的命中次数}] = S_n.$$

于是有 $S_0 = 0, S_1 = p$ 。在第 1 轮中, 要么 (i) 以概率 p 命中, 此时第 2 轮不能再命中 (因为在一次正确推测之后, 紧接着并不存在新的推测窗口), 贡献为 $1 + S_{n-2}$; 要么 (ii) 以概率 $1 - p$ 失误, 此后第 2 轮正常继续, 贡献为 S_{n-1} 。因此有如下递推:

$$S_n = p(1 + S_{n-2}) + (1 - p)S_{n-1}.$$

接下来把这个线性递推拆分为齐次部分和特解部分来求解。

(1) 齐次部分

$$S_n^{(h)} = pS_{n-2} + (1 - p)S_{n-1}.$$

其特征方程为

$$r^2 - (1 - p)r - p = 0 = (r - 1)(r + p) \implies \text{根为 } r_1 = 1, r_2 = -p.$$

因此

$$S_n^{(h)} = C_1 + C_2(-p)^n.$$

(2) 特解非齐次项是常数 $(+p)$, 且 $r = 1$ 是特征根, 所以常数试探解会与齐次部分冲突。令 $S_n^{(p)} = an$ 并代入:

$$an = (1 - p)a(n - 1) + pa(n - 2) + p = an - a(1 + p) + p,$$

由此得到 $a(1 + p) = p$, 于是

$$S_n^{(p)} = \frac{p}{1 + p} n.$$

合并得

$$S_n = C_1 + C_2(-p)^n + \frac{p}{1 + p} n.$$

利用 $S_0 = 0$ 与 $S_1 = p$:

$$0 = C_1 + C_2, \quad p = C_1 + C_2(-p) + \frac{p}{1 + p}.$$

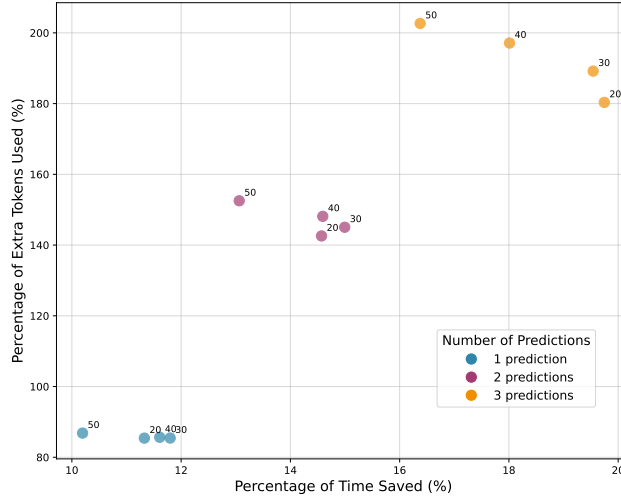


图 6: 在不同预测数与不同步数设置下, 额外 token 使用比例与节省时间比例之间的关系; 结果对 5 次运行取平均。

$$\text{解得 } C_2 = -\frac{p^2}{(1+p)^2}, \quad C_1 = \frac{p^2}{(1+p)^2}.$$

因此, 闭式解为

$$S_n = \frac{p}{1+p} n + \frac{p^2}{(1+p)^2} (1 - (-p)^n).$$

总节省。一共有 $T - 1$ 个可能的推测窗口, 因此

$$E[T_s] = \frac{T}{\beta} - S_{T-1} \frac{\alpha}{\beta(\alpha + \beta)}.$$

最终比值。两边除以 $E[T_{\text{seq}}] = T/\beta$, 得到

$$\frac{E[T_s]}{E[T_{\text{seq}}]} = 1 - \frac{1}{T} \frac{\alpha}{\alpha + \beta} \left[\frac{(T-1)p}{1+p} + \frac{p^2}{(1+p)^2} - \frac{p^2}{(1+p)^2} (-p)^{T-1} \right].$$

□

当 $T \rightarrow \infty$ 时, 上式恰好收敛到 $1 - \frac{p}{1+p} \frac{\alpha}{\alpha + \beta}$ 。

B 更多环境细节

B.1 国际象棋

节省时间与 token 成本之间的权衡。除了节省时间之外, 我们还衡量了并行推测带来的额外 token 成本。我们使用**额外 token 比例**这一指标来刻画它: $(M_{\text{sequential}} - M_{\text{speculative}})/M_{\text{sequential}}$ 。图 6 展示了不同预测数下节省时间与 token 成本的权衡关系。可以看到, 随着预测数增加, 额外 token 比例会上升, 而节省时间比例也会同步增加, 形成了清晰的 trade-off。

B.2 电商

τ -bench: 这是一个面向动态任务型对话的基准, 其中用户 (由语言模型模拟) 与一个具备 API 增强能力的智能体进行交互。该基准覆盖两个领域——零售与航空——并提供结构化数据库与领域特定工具。我们关注零售域, 在这个设置中, 智能体需要帮助用户完成取消或修改待处理订单、发起退货或换货、以及查询商品与订单信息等操作。该基准共定义了 115 个任务和 15 个 API (其中 7 个有写入副作用, 8 个为只读)。

预测准确率与成本之间的权衡。 图 7a 中的时间成本由延迟 (Time to First Token) 与输出响应时间共同组成。虚线表示平均用户打字时间, 按每分钟 40 个词估计。在这一阈值下, 多模型设置大约能达到 34% 的预测准确率, 这意味着在超过三分之一的情况下, 智能体可以无需等待 API 真正执行, 就立即给出响应。由此可见, 推测能够把原本可感知的迟滞体验, 转变为工具密集型环境下接近实时的交互。

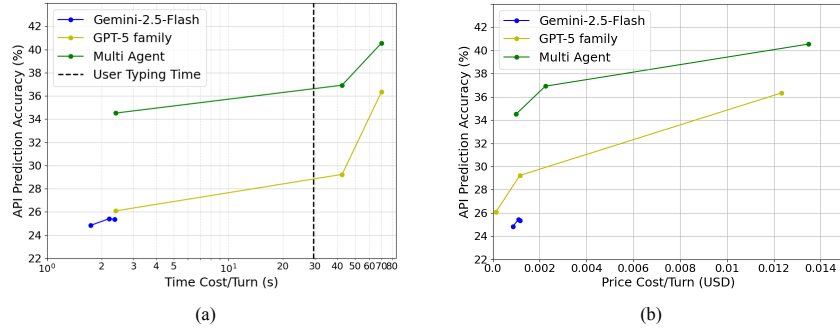


图 7: 不同模型下预测准确率与 Speculator 成本的关系。(a) 准确率与推测时间成本之间的权衡, 虚线表示平均用户打字时间; (b) 准确率与推测价格成本之间的权衡, 反映推测执行带来的货币开销。

B.3 操作系统调优

B.3.1 实验设置与实现细节

系统与工作负载配置 所有实验都在一台专用机器上进行: 2× Intel Xeon Silver 4114 10 核 CPU (2.20 GHz)、192 GB DDR4 内存、1 TB NVMe SSD, 运行 Ubuntu 22.04 与 Linux Kernel 5.15, 并托管在 Cloudlab 上 (Duplyakin et al., 2019)。

我们运行 `sysbench cpu` (Kopytov, 2020), 这是一个 CPU 密集型基准, 会反复计算一段大型素数序列。该基准会每秒报告多项性能指标。我们让 `sysbench` 在 16 个并发线程上运行, 并将其绑定到两个 CPU 核上。

调优器实现细节 系统由两个智能体组成: 一个快速的 Speculator 和一个缓慢的 Actor; 它们协同工作以最小化工作负载的 p95 延迟。在每一步中, 调优器提出一个新配置, 并立刻应用到在线系统上。应用这些参数本身几乎是瞬时的。

CFS 参数细节 完全公平调度器 (CFS) 是 Linux 的一个 CPU 调度器, 目标是让每个任务都能公平地获得 CPU 时间。它暴露了多个可调超参数供管理员调整其行为。我们调优的参数是 `min_granularity_ns`, 它规定一个任务在被抢占前至少能获得的时间片长度。提示词模板引导智能体在 50,000 到 50,000,000 纳秒 (0.05 ms 到 50 ms) 的范围内探索。Kernel 5.15 下该参数的默认值是 3 ms。一般预期是: 较小的参数值可以提升响应性, 但会增加上下文切换开销; 较大的参数值则有利于吞吐, 却可能恶化延迟。

历史压缩 历史压缩通过为两个智能体设计不同的提示结构来实现。当较慢的 Actor 被调用时, 其提示上下文中只包含一个压缩后的汇总, 概括了它思考窗口内发生的全部动作。Speculator 采取的每个动作都被压缩成一个简短的 (参数, 结果) 对。相比之下, 较快的 Speculator 接收到的是混合上下文: 它既看到上一个 Actor 周期留下的同样压缩历史, 也看到自己最近若干次动作的完整、详细回复。这种双上下文机制使 Actor 能够基于紧凑摘要分析长期趋势, 而 Speculator 则保留了短期、细粒度上下文, 以支持快速响应式决策。

B.3.2 多智能体优化中的提示工程

下面给出用于引导两个 LLM 智能体的提示词模板。

Actor 与 Speculator 的初始系统提示词

你是一名 Linux 内核调度器调优专家，对完全公平调度器（CFS）有深入理解。

多智能体角色：你是一个多智能体系统的一部分。

[针对 Actor]

你是 Actor。你的职责是给出经过充分分析、深思熟虑的参数建议。你会与一个快速探索参数空间的 Speculator 协同工作。你将接收到来自多次智能体调用的累计结果，以便进行更深入的分析并识别趋势。

[针对 Speculator]

你是 Speculator。你的职责是在每个时间窗口里立刻给出直觉式参数建议。你会与一个执行更深层分析的 Actor 协同工作。

你的目标是**最小化**一个 CPU 密集型工作负载的 p95 延迟。工作负载性能指标可能带有**噪声**，因此请寻找不同配置之间一致性的趋势。

可调的 CFS 参数：

- **min_granularity_ns**：抢占前的最小切片。值越小，响应性越高，但开销也越大；值越大，吞吐通常更好，但延迟可能更差。

参数范围：

- **min_granularity_ns**：50,000 到 50,000,000 纳秒

未来的调用中会提供性能数据。请**仅**按照如下格式回答：

Analysis: < 用 1 到 2 句话说明你的决策理由 >

Config: { "min_granularity_ns": <int> }

给 Speculator 的更新提示

[上下文包含第 1-10 次调用的压缩历史，以及第 11-18 轮迭代中 Speculator 的原始回复]

当前最佳：p95 latency=[value]，出现在第 #[value] 次调用

第 #19 次调用的最新结果：

Config: "min_granularity_ns": [value] → p95 latency=[value]

请给出你的分析，以及第 #20 轮迭代的下一个配置。

给 Actor 的更新提示

[上下文包含第 1-10 次调用的压缩历史]

当前最佳：p95 latency=[value]，出现在第 #[value] 次调用

第 #11 次调用结果 [SPECULATOR]: min_granularity_ns=[value] → p95 latency=[value]

第 #12 次调用结果 [SPECULATOR]: min_granularity_ns=[value] → p95 latency=[value]

...

第 #19 次调用结果 [SPECULATOR]: min_granularity_ns=[value] → p95 latency=[value]

请分析这一趋势，并给出第 #20 次调用的下一个配置。

示例智能体回复

Analysis: 性能在 300,000 ns 左右达到峰值，因此最优值很可能位于这一带。我将围绕该峰值继续缩小搜索范围。

Config: { "min_granularity_ns": 250000 }

B.3.3 Token 使用量与成本

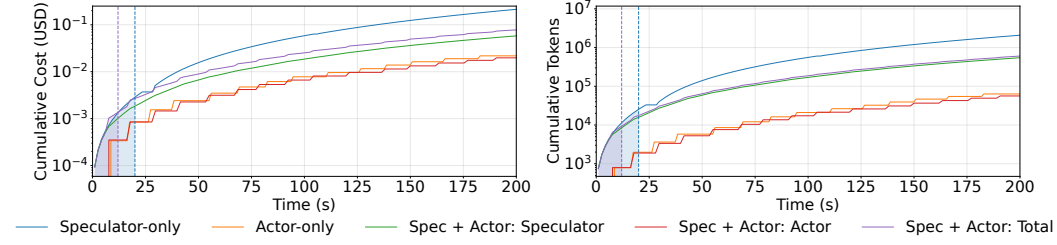


图 8: 累计 token 使用量与成本随时间的变化。左图与右图分别展示了三种配置下累计成本 (美元) 与累计 token 使用量。竖线标出了各系统观测到的收敛时刻。

表 2: 若干时间点上的累计 token 数与成本 (单位: 美分)。

已过时间	仅 Actor		仅 Speculator		Actor+Speculator (总计)	
	Tokens	成本 (美分)	Tokens	成本 (美分)	Tokens	成本 (美分)
初始	744	0.02	690	0.01	690	0.03
10s	790	0.03	9,539	0.11	8,548	0.13
30s	3,631	0.15	45,768	0.57	32,459	0.48
60s	8,581	0.35	205,794	2.24	84,568	1.18
120s	26,398	0.96	778,253	8.12	261,855	3.53
200s	63,376	2.18	2,099,894	21.5	607,877	7.83

如图 8 所示,并结合表 2 中的数据可以看到, Speculator 的高频调用会导致 token 消耗与成本快速增长。不过在实践中,这种增长会被系统更快的收敛所限制。联合的 Actor-Speculator 系统大约在 15 秒时收敛,而仅 Speculator 系统大约在 20 秒时收敛;仅 Actor 系统则要到 200 秒后才收敛。一旦最优状态达到,调优过程就可以结束,因此在该场景下,长期指数式成本增长的风险实际上可以忽略。未来还可以通过若干优化策略进一步抑制 token 增长,例如把上下文截断为固定窗口,或者在收敛后关闭探索。

B.3.4 推测带来的响应时间收益

为了更有针对性地展示推测如何缓解瞬时性能损失,我们进行了一个受控实验。在该实验中,系统会在时刻 t_0 被人为扰动:把 `min_granularity` 参数设置为一个高度次优的值 (10 ms)。随后,我们比较两种配置下系统的恢复过程:一种是完整的 Actor-Speculator 系统;另一种是仅 Actor 基线,它只重放完整 Actor-Speculator 轨迹中由 Actor 提出的那些动作。

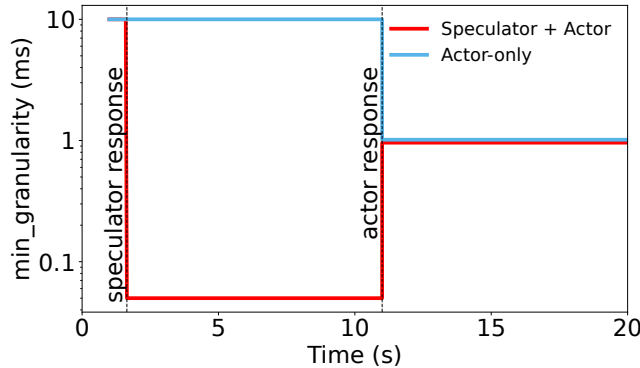


图 9: 一个受控实验: 系统在 $t = 0$ 时受到人为扰动后的阶跃响应。**Actor-Speculator** 系统在 1 秒内纠正了糟糕设置,而**仅 Actor** 系统则必须等待 10 秒以上,才能进入下一次决策周期。该实验的定量结果在正文的图 5 (右) 中进行了汇总。

如图 9 所示，Actor-Speculator 系统几乎会立刻作出反应。快速的 Speculator 一旦观察到性能突然恶化，就会立即采取修正动作，使系统大约在 1 秒内恢复到高效状态。相比之下，仅 Actor 系统不得不承受 10 秒以上的糟糕性能，因为它必须等较慢的 Actor 完成整轮思考之后才能采取行动。图中展示的性能差距，已在正文中（图 5，右）作了量化说明。